DevOps Pipeline for the H5X Website

DevOps Lab Project Report

Bachelor of Technology

Computer Science and Engineering

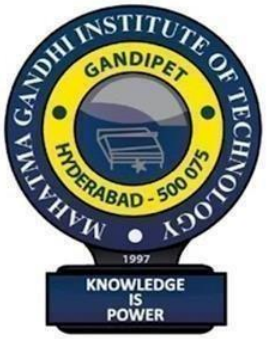
By

Md. Faisal Tabrez (23261A0535)

Under the guidance of

K. Rajitha

Associate Professor

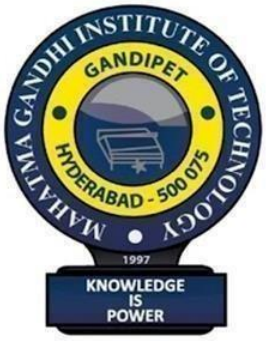


DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY

GANDIPET, HYDERABAD-500075, TELANGANA

CERTIFICATION



This is to certify that the DevOps Lab project entitled **DevOps Pipeline for the H5X Website** is being submitted by **Md. Faisal Tabrez**, bearing **Roll No. 23261A0535**, in partial fulfilment of the requirements for the award of **Bachelor of Technology in Computer Science and Engineering** at **Mahatma Gandhi Institute of Technology**. This report records the project implementation, execution outputs and analysis carried out under the guidance and supervision of the project guide.

Project Guide
K. Rajitha
Associate Professor
Department of CSE

Head of Department
Department of CSE

DECLARATION

I, **Md. Faisal Tabrez (23261A0535)**, declare that this DevOps Lab project report, **DevOps Pipeline for the H5X Website**, presents the DevOps implementation and analysis recorded on the repository branch **devops/h5x-lab-project**. The existing H5X website, its credited upstream material and the software tools used in this work are acknowledged. The terminal outputs, browser screenshots and pipeline results in this report are based on executed commands and saved artifacts.

The Kubernetes results describe a local single-node teaching environment. They demonstrate deployment, manual scaling and pod replacement; they do not establish production availability or automatic scaling.

Student signature: _____

Md. Faisal Tabrez (23261A0535)

ACKNOWLEDGEMENT

I thank **K. Rajitha, Associate Professor, Department of Computer Science and Engineering**, for guidance and support. I also thank the department faculty and staff of Mahatma Gandhi Institute of Technology. I acknowledge the maintainers of Git, GitHub, Jenkins, Docker, Selenium, Kubernetes and kind, and the contributors whose work is credited in the H5X repository.

Md. Faisal Tabrez (23261A0535)

TABLE OF CONTENTS

Section	Page
Certification	2
Declaration and Acknowledgement	3
1 Introduction	5
2 Git and GitHub	6
2 GitHub Actions execution	7
3 Jenkins pipeline	8
3 Jenkins command outputs	9
4 Docker	10
5 Selenium tests	11
5 Selenium browser output	12
6 Kubernetes deployment	13
6 Kubernetes scaling and recovery	14
7 Conclusion and References	15

Project repository
github.com/FaisalTabrez/H5X/tree/devops/h5x-lab-project

Implementation revision
5c4d3af21c37617d5e3643ba0185b0d466ad8d2b

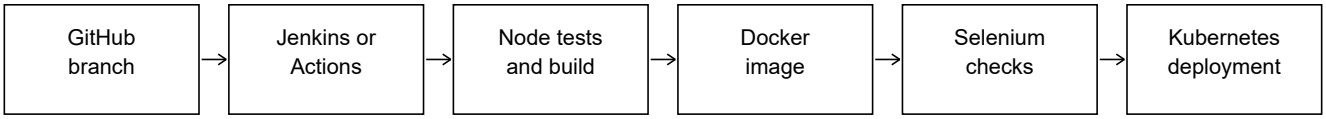
Evidence record
Jenkins H5X-DevOps build 2; GitHub Actions run 42673103294, attempt 2. Execution logs use UTC timestamps. Full logs and original-resolution screenshots accompany the report in devops/evidence.

1 Introduction

This project implements a repeatable delivery pipeline for the H5X team website. H5X contains five pages for CTF activities, bug hunting, hackathons, events and photographs. The application builds with Node.js and produces static HTML, CSS, JavaScript and image assets. The DevOps work adds automated checks, container packaging, browser acceptance tests and a real Kubernetes deployment.

The objective is to verify a change before deployment and retain evidence of the result. Jenkins runs the local delivery pipeline, while GitHub Actions independently verifies the branch on a hosted runner. Both execute the website build, Docker packaging, Selenium tests and local Kubernetes rollout. The checked-in CTFtime snapshot keeps the build independent of live data availability.

Pipeline flow



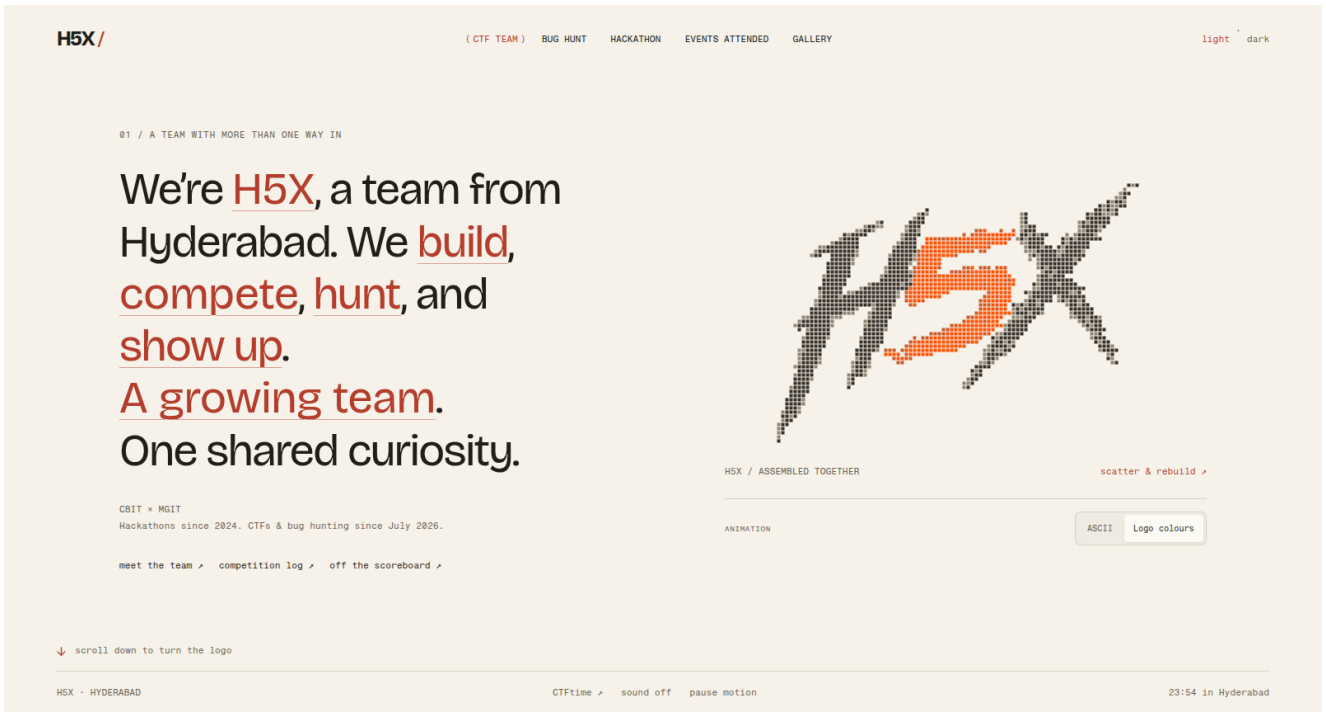


Figure 1. The H5X website served by the local Kubernetes Service.

Component	Lab implementation
Application	Five static pages; no external database
Execution environment	Docker Desktop Linux containers; kind v0.33.0
Deployment target	Kubernetes v1.37.0; namespace h5x-devops

2 Git and GitHub

The implementation is committed to **devops/h5x-lab-project** in the existing H5X repository. Git records the changes to the pipeline, Dockerfile, browser tests and Kubernetes manifests. Jenkins fetches this branch and checks out its exact source revision, so its workspace appears as a detached HEAD; the remote branch shown below identifies the revision being tested.

```
git switch -c devops/h5x-lab-project
git add .
git commit -m "Add DevOps lab pipeline with Docker Selenium and Kubernetes"
git push -u origin devops/h5x-lab-project
```

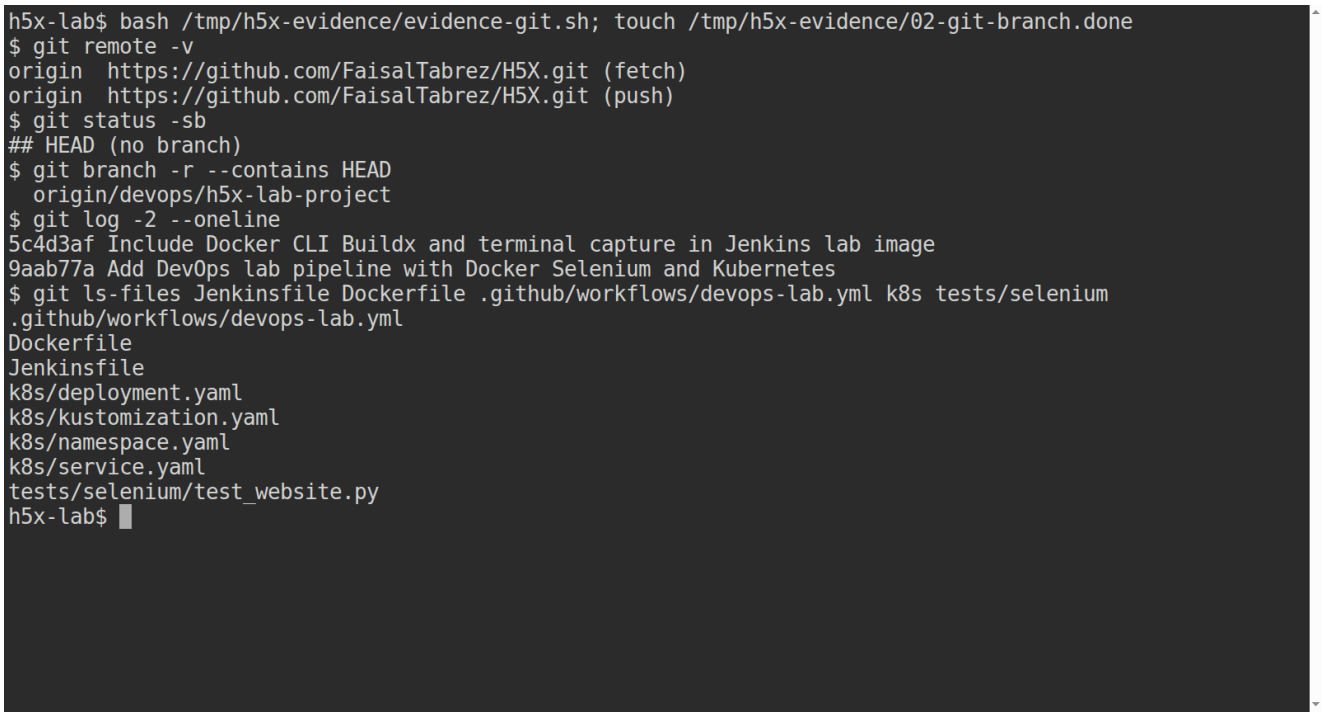


Figure 2. Real Git terminal output from the Jenkins workspace.

Tracked file or folder	Purpose
Jenkinsfile	Five-stage local CI/CD pipeline
Dockerfile and devops/nginx.conf	Build and serve the website container
tests/selenium/	Browser acceptance checks
k8s/ and devops/kind.yaml	Deployment, Service, namespace and local cluster
.github/workflows/devops-lab.yml	Hosted verification and saved artifacts

2 GitHub Actions execution

The workflow ran against revision **5c4d3af**. Run **42673103294, attempt 2**, completed successfully, including unit tests, Docker build, eight Selenium checks, cluster creation and deployment verification. The first attempt of this run stopped with a GitHub internal server error before any job started; the retry used the same source revision.

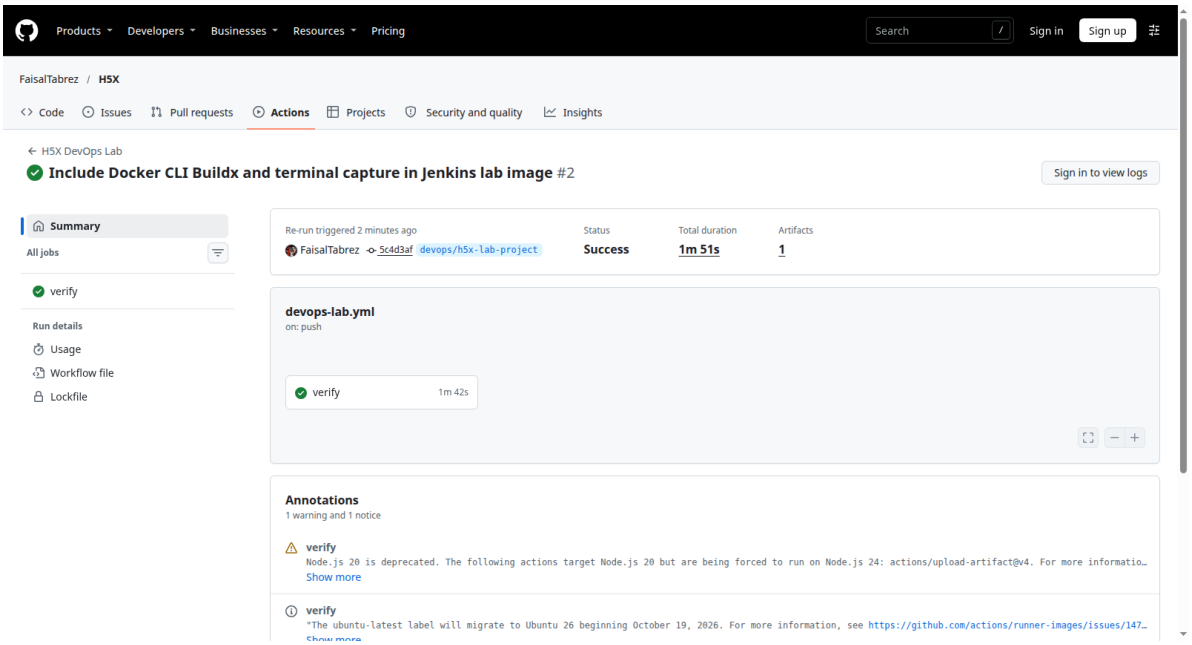


Figure 3. GitHub Actions showing the successful verify job.

```
h5x-lab$ bash /tmp/h5x-evidence/evidence-actions.sh; touch /tmp/h5x-evidence/13-github-actions-output.don
e
GitHub Actions command output recorded on the Windows host
Command: gh run view 42673103294 --repo FaisalTabrez/H5X --log

Selected test and deployment lines; full log accompanies the report.

2026-11-09T18:23:14.2035120Z # tests 14
2026-11-09T18:23:14.2036347Z # pass 14
2026-11-09T18:23:14.2036970Z # fail 0
2026-11-09T18:23:49.9814469Z collecting ... collected 8 items
2026-11-09T18:24:01.9602209Z ============================== 8 passed in 12.10s ========================
====
2026-11-09T18:24:43.2672727Z namespace/h5x-devops created
2026-11-09T18:24:43.2768749Z service/h5x created
2026-11-09T18:24:43.2897523Z deployment.apps/h5x created
2026-11-09T18:24:47.8556654Z deployment "h5x" successfully rolled out
2026-11-09T18:24:47.9568841Z deployment.apps/h5x   2/2     2           2           4s    website      h5
x-devops:ci   app=h5x
2026-11-09T18:24:47.9675126Z healthy
h5x-lab$
```

Figure 4. Selected test and deployment output from the hosted Actions job.

The workflow uploads JUnit XML, browser screenshots and Kubernetes manifests as **h5x-devops-evidence**. The terminal displays selected command-output lines from the hosted job. Complete logs, including runner notices, are preserved in **devops/evidence/logs/github-actions.log**.

3 Jenkins pipeline

A separate Jenkins controller was run in Docker for the lab. Its initialization creates the **H5X-DevOps** Pipeline job and loads the Jenkinsfile from the project branch. Build **2** completed with **SUCCESS** and archived the test XML, browser images and rendered deployment manifest.

Stage	Executed work
Git checkout	Fetch the branch and record revision 5c4d3af
Test and build	Run npm test, build five pages and validate assets
Docker image	Build h5x-devops:build-2 and inspect its user
Selenium acceptance	Run eight browser checks against a temporary container
Kubernetes deploy	Load the image into kind, apply manifests and verify health

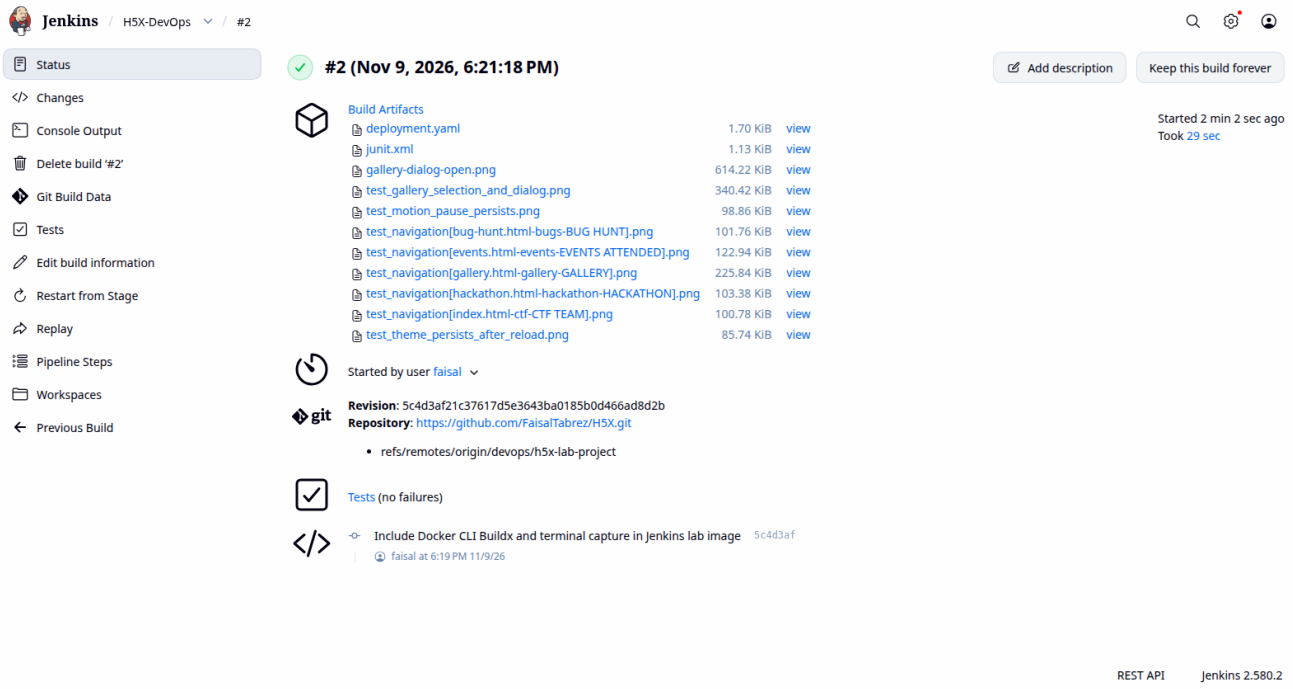


Figure 5. Jenkins build 2 with successful status, source revision and artifacts.

The lab controller includes Node.js, Python, Docker CLI with Buildx, kind and kubectl. It uses one executor and prevents overlapping runs of this job. The Docker socket and cluster credentials are available only to this trusted local lab; passwords and kubeconfigs are excluded from Git.

3 Jenkins command outputs

The existing Node test suite passed all **14 tests**. The static build generated five pages, and the site checker verified their assets, records and gallery credits. The terminal screenshot repeats these same project commands in the checked-out Jenkins workspace.

```
h5x-lab$ bash /tmp/h5x-evidence/evidence-node.sh; touch /tmp/h5x-evidence/06-node-tests-build.done
$ npm test
$ tail -10
  ...
1..14
# tests 14
# suites 0
# pass 14
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 130.709682
$ npm run build:pages

> build:pages
> node scripts/build-h5x.mjs --pages

Built five H5X pages in /var/jenkins_home/workspace/H5X-DevOps/out using the researched local snapshot.
$ npm run check:site

> check:site
> node scripts/check-site.mjs

Five pages, project-relative assets, H5X identity, valid records and gallery exclusions verified.
h5x-lab$
```

Figure 6. Node test summary, static build and site validation output.

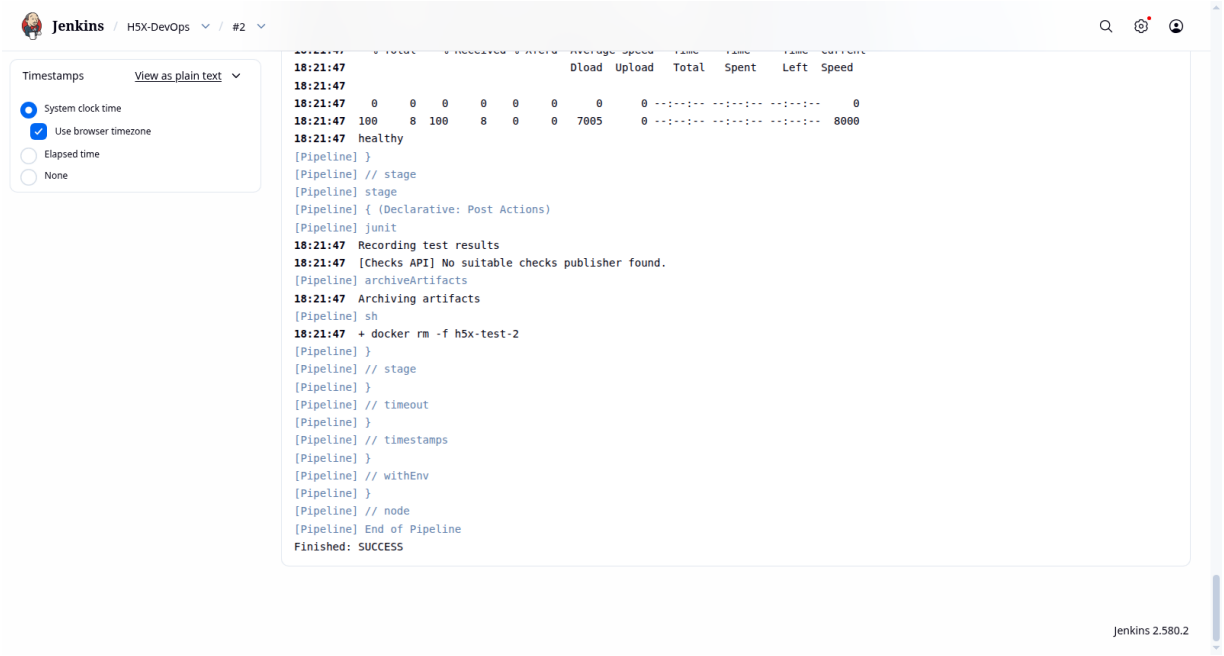


Figure 7. Jenkins console ending with Finished SUCCESS.

Build 1 exposed a missing Docker CLI in the original controller image. The controller Dockerfile was corrected to include Docker CLI and Buildx, then build 2 passed. The archived console log records the successful run without substituting expected output for actual results.

4 Docker

The multi-stage Dockerfile builds the website with Node.js and copies only the generated output into unprivileged Nginx. The runtime uses user **101** and listens on port **8080**. The /healthz endpoint supports container health checks and Kubernetes probes. The measured local image size was **93,283,132 bytes**.

```
FROM node:22-alpine AS build
WORKDIR /app
# Copy package.json, scripts, src, static and data
RUN npm run build:pages && npm run check:site
FROM nginxinc/nginx-unprivileged:stable-alpine
COPY devops/nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/out/ /usr/share/nginx/html/
EXPOSE 8080
```

The excerpt summarizes the two build stages; the repository contains the complete Dockerfile, source-copy instructions and HEALTHCHECK.

```
h5x-lab$ bash /tmp/h5x-evidence/evidence-docker.sh; touch /tmp/h5x-evidence/07-docker-build-health.done
$ docker build --progress=plain -t h5x-devops:local .
$ tail -7
#17 exporting config sha256:6baf5cff736347c4597a2cd0874920bbf135c5628eef8c1767d387b6ca2cb811 done
#17 exporting attestation manifest sha256:d857f1a1153018450a2e2f8ff4df6ce10d4637a820677e0db758e5cc843ae53
c
#17 exporting attestation manifest sha256:d857f1a1153018450a2e2f8ff4df6ce10d4637a820677e0db758e5cc843ae53
c 0.0s done
#17 exporting manifest list sha256:57150910d5598869d9fc9362fe1ca477f7dcf0a11dea8c4ce3f23aed5a25dffa 0.0s
done
#17 naming to docker.io/library/h5x-devops:local done
#17 unpacking to docker.io/library/h5x-devops:local 0.0s done
#17 DONE 0.1s
$ docker image inspect h5x-devops:local --format 'User={{.Config.User}} Size={{.Size}} bytes'
User=101 Size=93283132 bytes
$ docker ps --filter name=h5x-demo --format 'table {{.Names}}\t{{.Status}}'
NAMES      STATUS
h5x-demo   Up 5 minutes (healthy)
$ docker inspect h5x-demo --format 'Health={{.State.Health.Status}}'
Health=healthy
$ curl -fsS http://h5x-demo:8080/healthz
healthy
$ curl -s -o /dev/null -w 'Home HTTP %{http_code}\n' http://h5x-demo:8080/
Home HTTP 200
$ curl -s -o /dev/null -w 'Unknown route HTTP %{http_code}\n' http://h5x-demo:8080/missing
Unknown route HTTP 404
h5x-lab$
```

Figure 8. Docker build, non-root user, healthy container and HTTP checks.

The home route returned **HTTP 200**, the missing route returned **HTTP 404**, and the running container reported **healthy**. The Jenkins image tag build-2 identifies the container image loaded into Kubernetes.

5 Selenium tests

Python Selenium WebDriver controls Chromium through a remote Selenium server. Pytest runs independent browser sessions, explicit waits synchronize with visible controls, and screenshots are saved after each test. The same acceptance suite is used locally, in Jenkins and in GitHub Actions.

Check	Cases	Verified behaviour
Navigation	5	Each menu link opens the correct page, title and active item
Theme persistence	1	Dark mode survives reload; light mode can be restored
Gallery interaction	1	Photo selection updates; full-image dialog opens and closes
Motion persistence	1	Pause state and accessibility state survive reload

```
h5x-lab$ bash /tmp/h5x-evidence/evidence-selenium.sh; touch /tmp/h5x-evidence/08-selenium-tests.done
$ /opt/browser-tests/bin/python -m pytest tests/selenium -v --junitxml=reports/evidence-tests.xml
============================= test session starts ==============================
platform linux -- Python 3.13.5, pytest-8.4.2, pluggy-1.7.0 -- /opt/browser-tests/bin/python
cachedir: .pytest_cache
rootdir: /var/jenkins_home/workspace/H5X-DevOps
collecting ... collected 8 items

tests/selenium/test_website.py::test_navigation[index.html-ctf-CTF TEAM] PASSED [ 12%]
tests/selenium/test_website.py::test_navigation[bug-hunt.html-bugs-BUG HUNT] PASSED [ 25%]
tests/selenium/test_website.py::test_navigation[hackathon.html-hackathon-HACKATHON] PASSED [ 37%]
tests/selenium/test_website.py::test_navigation[events.html-events-EVENTS ATTENDED] PASSED [ 50%]
tests/selenium/test_website.py::test_navigation[gallery.html-gallery-GALLERY] PASSED [ 62%]
tests/selenium/test_website.py::test_theme_persists_after_reload PASSED  [ 75%]
tests/selenium/test_website.py::test_gallery_selection_and_dialog PASSED [ 87%]
tests/selenium/test_website.py::test_motion_pause_persists PASSED        [100%]

- generated xml file: /var/jenkins_home/workspace/H5X-DevOps/reports/evidence-tests.xml -
============================== 8 passed in 8.55s ===============================
h5x-lab$
```

Figure 9. Actual Selenium acceptance output showing eight passing tests.

The captured terminal execution completed with **8 passed in 8.55 seconds**. This is one observed test duration, not a performance benchmark. JUnit reports and the browser screenshots are saved as build artifacts; the hosted workflow independently passed the same eight checks.

5 Selenium browser output

The gallery test selects the second photograph using the visible thumbnail control, opens the full photograph and verifies that the dialog title matches the selected image. It then closes the dialog and waits for it to disappear. The browser image below was captured from the deployed Kubernetes website.



Figure 10. The real gallery dialog opened by Selenium on the deployed website.

Representative test assertions

```
wait.until(EC.element_to_be_clickable(
    (By.CSS_SELECTOR, '[data-photo="1"]'))).click()
title = browser.find_element(By.ID, "featured-title").text
browser.find_element(By.ID, "expand-photo").click()
wait.until(EC.visibility_of_element_located(
    (By.ID, "photo-dialog")))
assert browser.find_element(By.ID, "photo-title").text == title
browser.find_element(
    By.CSS_SELECTOR, "#photo-dialog .dialog-close").click()
```

The suite tests the website interface and local application state. It does not submit content to external sites or depend on successful navigation to the photographs' original social posts. The source tests are available in **tests/selenium/test_website.py**.

6 Kubernetes deployment

kind creates a real single-node Kubernetes cluster inside Docker. Jenkins loads its tested image into the node, renders the Kustomize resources, replaces the local image tag with **h5x-devops:build-2** and applies the manifest. The Deployment maintains two replicas; a NodePort Service exposes port 30080, mapped to host port 18088.

```
kind load docker-image h5x-devops:build-2 --name h5x-devops
kubectl --context kind-h5x-devops apply -f reports/manifests/deployment.yaml
kubectl -n h5x-devops rollout status deployment/h5x
curl --fail http://127.0.0.1:18088/healthz
```

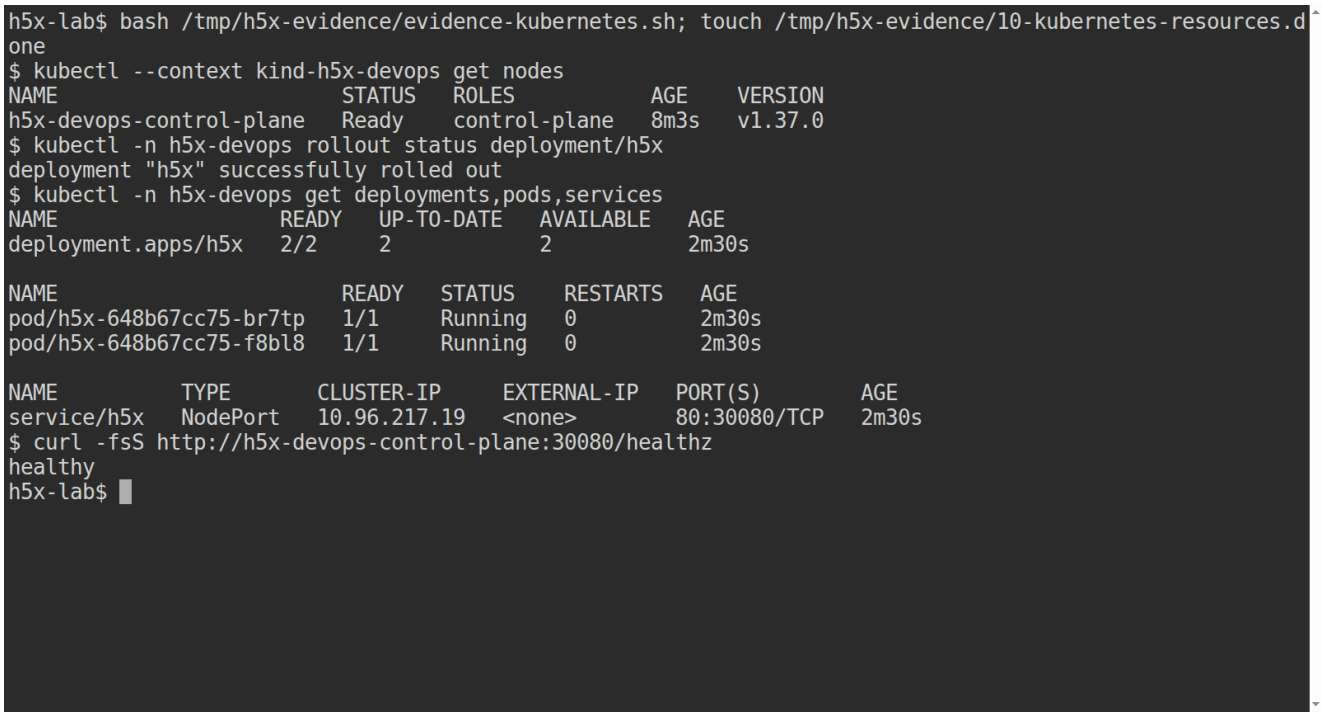


Figure 11. Ready Kubernetes node, two replicas, NodePort Service and health response.

Configuration	Applied value
Health checks	Readiness and liveness probes on /healthz
Resources per pod	Requests 50m CPU and 32Mi; limits 250m CPU and 128Mi
Security	Non-root user 101; read-only root; capabilities dropped
Update policy	Rolling update with maxUnavailable 0 and maxSurge 1

6 Kubernetes scaling and recovery

The deployment was scaled from two to three replicas. After all three pods became ready, one selected pod was deleted. Kubernetes created a replacement pod, readiness checks passed, and the deployment was restored to two replicas. These are executed cluster operations.

```
h5x-lab$ bash /tmp/h5x-evidence/evidence-scaling.sh; touch /tmp/h5x-evidence/11-kubernetes-scaling.done
$ kubectl -n h5x-devops scale deployment/h5x --replicas=3
deployment.apps/h5x scaled
$ kubectl -n h5x-devops rollout status deployment/h5x --timeout=90s
Waiting for deployment "h5x" rollout to finish: 2 of 3 updated replicas are available...
deployment "h5x" successfully rolled out
$ kubectl -n h5x-devops get pods
NAME                   READY   STATUS    RESTARTS   AGE
h5x-648b67cc75-br7tp   1/1     Running   0          3m24s
h5x-648b67cc75-f8bl8   1/1     Running   0          3m24s
h5x-648b67cc75-wfhsp   1/1     Running   0          4s
h5x-lab$
```

Figure 12. Scaling to three replicas and waiting for rollout completion.

```
h5x-lab$ bash /tmp/h5x-evidence/evidence-recovery.sh; touch /tmp/h5x-evidence/12-kubernetes-recovery.done
$ kubectl -n h5x-devops delete pod h5x-648b67cc75-br7tp --wait=true
pod "h5x-648b67cc75-br7tp" deleted from h5x-devops namespace
$ kubectl -n h5x-devops wait --for=condition=ready pod -l app=h5x --timeout=90s
pod/h5x-648b67cc75-f8bl8 condition met
pod/h5x-648b67cc75-qwt76 condition met
pod/h5x-648b67cc75-wfhsp condition met
$ kubectl -n h5x-devops get pods
NAME                   READY   STATUS    RESTARTS   AGE
h5x-648b67cc75-f8bl8   1/1     Running   0          3m34s
h5x-648b67cc75-qwt76   1/1     Running   0          4s
h5x-648b67cc75-wfhsp   1/1     Running   0          14s
$ kubectl -n h5x-devops scale deployment/h5x --replicas=2
deployment.apps/h5x scaled
$ kubectl -n h5x-devops rollout status deployment/h5x --timeout=90s
deployment "h5x" successfully rolled out
$ kubectl -n h5x-devops get deployment h5x
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
h5x    2/2     2            2           3m34s
h5x-lab$
```

Figure 13. Pod replacement, readiness checks and restoration to two replicas.

Deleted pod: **h5x-648b67cc75-br7tp**. Replacement pod: **h5x-648b67cc75-qwt76**. The final Deployment reported **READY 2/2**. This demonstrates replica reconciliation after pod deletion. It does not measure request continuity or recovery from failure of the single cluster node.

7 Conclusion

The project demonstrates a working DevOps pipeline for an existing website. Version-controlled configuration drives automated checks, a tested Docker image, browser acceptance tests and a verified Kubernetes rollout. Jenkins and GitHub Actions independently completed the delivery workflow for implementation revision **5c4d3af**.

Requirement	Observed result
Git and GitHub	Implementation branch pushed with tracked configuration
Jenkins	H5X-DevOps build 2 finished SUCCESS
Docker	Image built; user 101; container healthy; HTTP 200/404 checks
Selenium	Eight acceptance tests passed
Kubernetes	Two ready replicas; scaling to three; pod replacement verified
GitHub Actions	Run 42673103294 attempt 2 completed successfully

The deployment is a single-node local lab with manual scaling. Production work would require separate Jenkins agents, managed credentials, immutable image references, TLS, monitoring and a multi-node deployment. The captured results do not establish autoscaling, production high availability or performance improvements.

References

1. Project implementation and evidence
https://github.com/FaisalTabrez/H5X/tree/devops/h5x-lab-project

2. Successful GitHub Actions run
https://github.com/FaisalTabrez/H5X/actions/runs/42673103294

3. Jenkins documentation Using a Jenkinsfile
https://www.jenkins.io/doc/book/pipeline/jenkinsfile/

4. Docker documentation Multi-stage builds
https://docs.docker.com/build/building/multi-stage/

5. Selenium documentation Remote WebDriver
https://www.selenium.dev/documentation/webdriver/drivers/remote_webdriver/

6. Kubernetes documentation Deployments
https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

7. kind documentation Quick Start
https://kind.sigs.k8s.io/docs/user/quick-start/